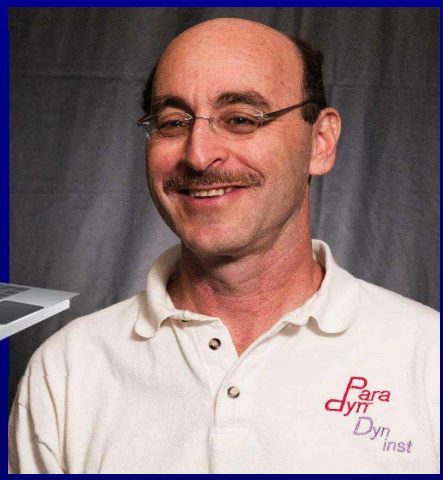
Introduction to Software Testing
Special Chapter
Fuzz Testing / Fuzzer / Fuzzing

Farn Wang

Dept. of EE, National Taiwan Univ.

Fuzzing / Fuzzer / Fuzz Testing

- Run program on many random, abnormal inputs and look for bad behavior in the responses
  - Bad behaviors such as crashes or hangs



Professor Barton P. Miller
Univ. of Wisconsin

- A stormy heavy-rain night in 1988
- Connected to office Unix system via a dial up connection
- Crashed many UNIX utilities

https://pages.cs.wisc.edu/~bart/fuzz/

Fuzzing / Fuzzer / Fuzz Testing

- Felt something deeper
- Coined the term

Fuzz testing

- Black-box approach
- Asked three groups in his grad-seminar course to implement this idea
- Two groups failed to achieve any crash results!
- The third group succeeded! Crashed 25-33% of the utility programs on the seven Unix variants that they tested

Professor Barton P. Miller

Univ. of Wisconsin

https://pages.cs.wisc.edu/~bart/fuzz/

Fuzzing / Fuzzer / Fuzz Testing

- A negative testing – finding errors
  - Fault injector
  - Errors found: not checking returns, Array indices out of bounds, not checking null pointers, …
  - In comparison: positive testing – liveness analysis,
    - checking good behaviors happen.
- *Automatic*
- Black-Box
  - now extended to gray-box, white-box
- malformed or semi-malformed test input

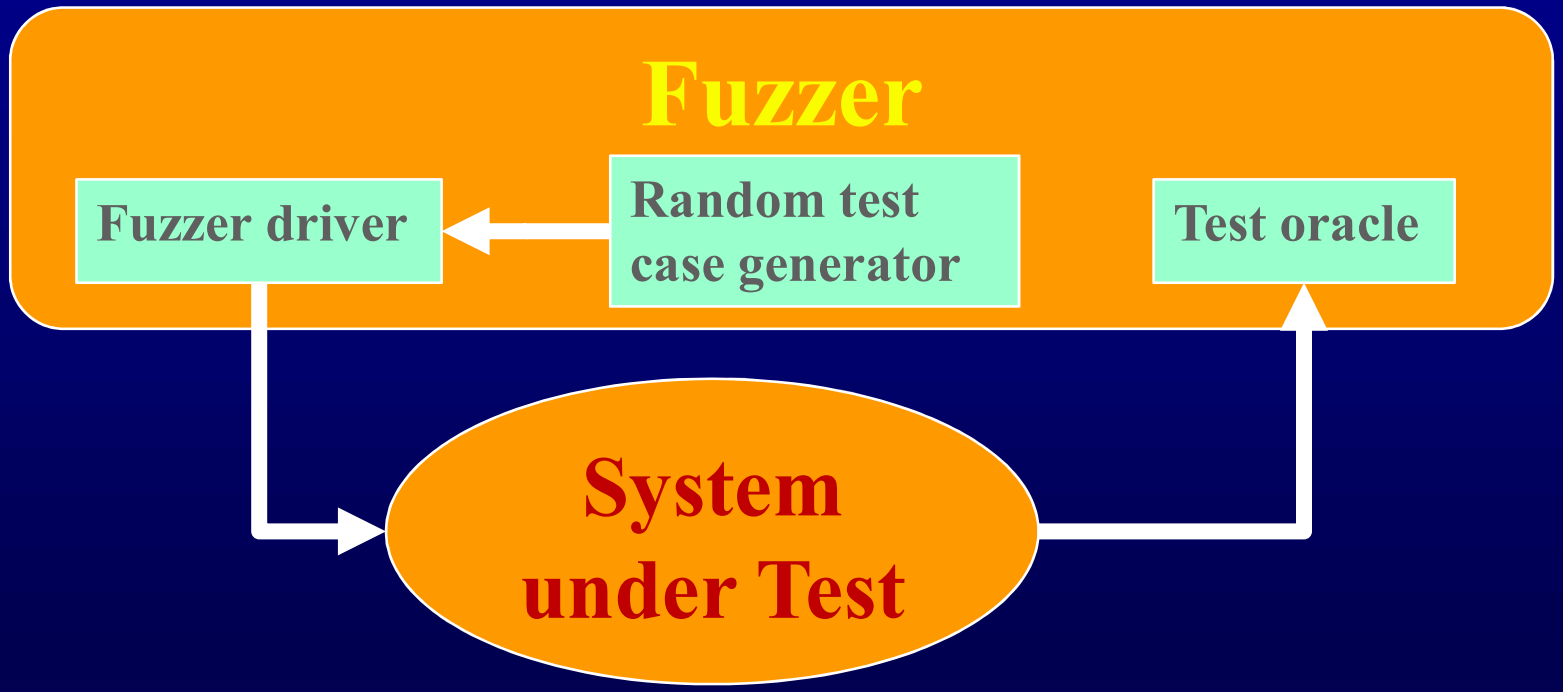
Fuzzing / Fuzzer / Fuzz Testing

So many terms in software/security testing

- Random Testing
  - Usually after formal testing (according to user/test requirements)
  - Find loopholes in an unbiased way.
  - Does not focus on bad input.
- Monkey Testing - acronym for random testing
- Gorilla testing – hundreds of times on a module
- Ad-hoc testing
  - Unstructured testing
- Fuzz Testing
  - Random testing with abnormal input

Fuzzer

- Run lots of programs using random inputs
- Identify failures of these programs
- Correlate random inputs with failures



Fuzzer Components

Random Test Case Generator

- Test input
  - Purely random impossible input.
    - ////////, httpssss://///, …
  - Forms
  - TCP/IP packets
  - Files with formats
  - Html
- Randomization
  - Purely random,
  - Feedback
    - Mutation, generator, Coverage-based in gray-box?

Test Oracles

- Crash dector
- Exception handling dumps
- Server response
- Assertion dumps in the code
- Oracles by users
- 3rd-party special purpose test oracle
  - Valgrind for memory faults?

Fuzzing Technologies

- **Purely random:**
  - **Some knowledge about the test input format**
  - **No feedback**
- **Block based:**
  - **Some functions to check the interface.**
- **Feedback-based:**
  - **Learning based on feedback loop.**
- **Model-based or simulation-based:**
  - **Correct behavior of the SuT (System under Test) is provided in a model or a simulator.**

Fuzzer
Steps in loop iteration

1. Identify target
2. Identify inputs
3. Generate fuzzed data
4. Execute fuzzed data
5. Monitor for exceptions
6. Determine exploitability

Example:

1.Relational database engine

2.SQL interface of the DBMS

3.…

4.Send SQL statements to server

5.Crashes, resource usage, etc?

Fuzzer: Types of SuT (System under Test)

- **Application**
  - **For desktop/mobile apps: the UI, the command-line options, the import/export capabilities.**
  - **For web apps: urls, forms, user-generated content, RPC requests**
- **Protocol fuzzing**
  - **Sending forged packets to application.**
  - **maybe as a proxy, modifying requests on the fly and replaying them.**
- **File format fuzzing**
  - **Generates multiple malformed samples, and opens them sequentially.**

Fuzzing

Simple examples on SMTP

- **mail from: sender@testhost**
  - **This would then be sent in the following forms to see what effect they have:**
- **mailmailmailmail from: sender@testhost**
- **mail fromfromfromfrom: sender@testhost**
- **mail from:::: sender@testhost**
- **mail from: sendersendersendersender@testhost**
- **mail from: sender@@@@testhost**
- **mail from: sender@testhosttesthosttesthosttesthost**

Fuzzing

Standard HTTP GET request

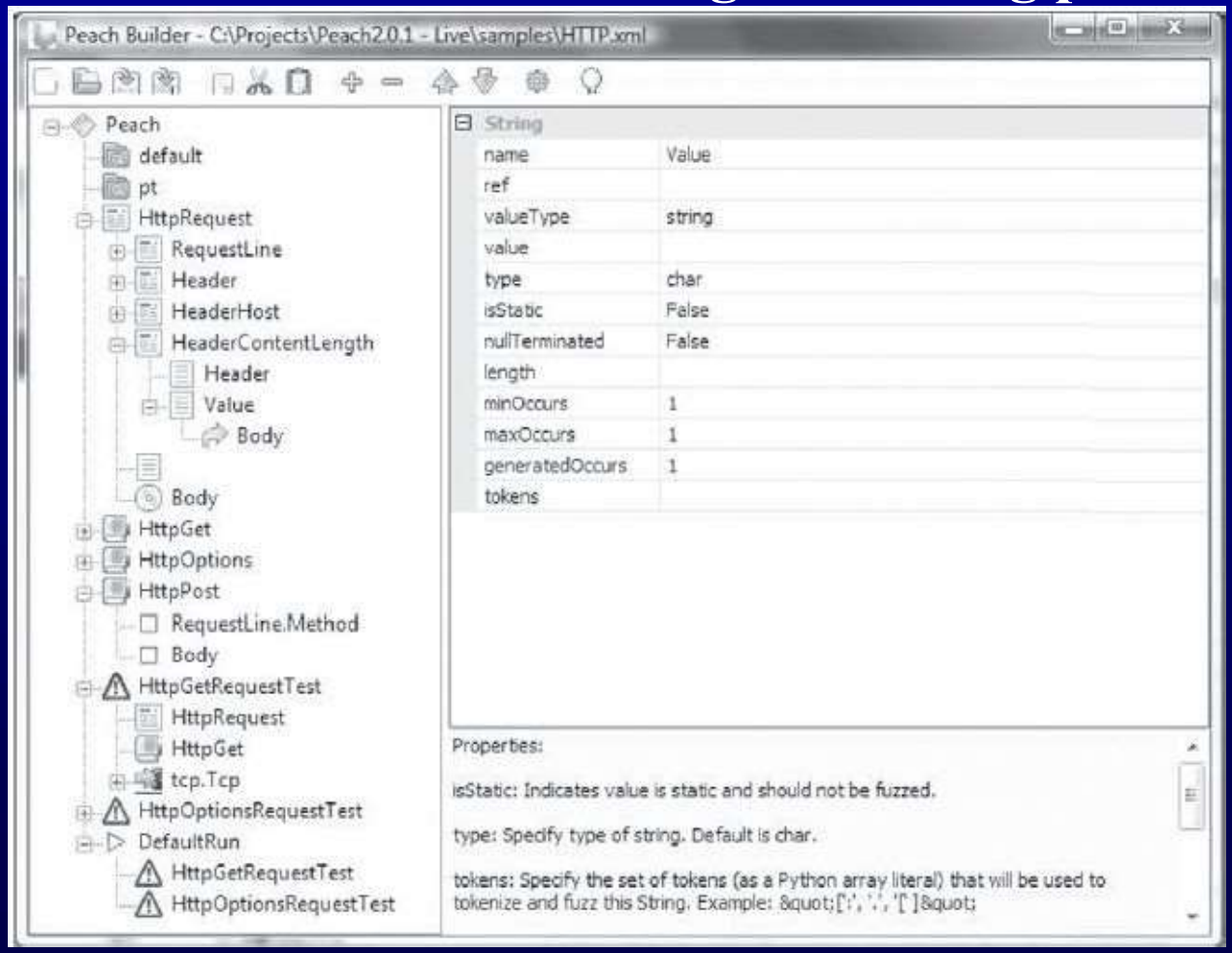
- **GET /index.html HTTP/1.1**

Anomalous requests

- **AAAAAA...AAAA /index.html HTTP/1.1**
- **GET ///////index.html HTTP/1.1**
- **GET %n%n%n%n%n%n.html HTTP/1.1**
- **GET /AAAAAAAAAAAAA.html HTTP/1.1**
- **GET /index.html HTTTTTTTTTTTTTP/1.1**
- **GET /index.html HTTP/1.1.1.1.1.1.1.1**

Fuzzing a protocol

Peach is a Smart Fuzzer that is capable of generating a protocol from scratch or mutating an existing protocol.



Fuzzing a file

Example: A Sample PNG file spec.

```
//png.spk
//author: Charlie Miller

// Header - fixed.
s_binary("89504E470D0A1A0A");

// IHDRChunk
s_binary_block_size_word_bigendian("IHDR"); //size of data field
s_block_start("IHDRcrc");
        s_string("IHDR");  // type
        s_block_start("IHDR");
// The following becomes s_int_variable for variable stuff
// 1=BINARYBIGENDIAN, 3=ONEBYE
                s_push_int(0x1a, 1);     // Width
                s_push_int(0x14, 1);     // Height
                s_push_int(0x8, 3);      // Bit Depth - should be 1,2,4,8,16, based
   on colortype
                s_push_int(0x3, 3);      // ColorType - should be 0,2,3,4,6
                s_binary("00 00");       // Compression || Filter - shall be 00 00
                s_push_int(0x0, 3);      // Interlace - should be 0,1
        s_block_end("IHDR");
s_binary_block_crc_word_littleendian("IHDRcrc"); // crc of type and data
s_block_end("IHDRcrc");
...
```

Fuzzing emails

Simple examples on SMTP

- **mail from: sender@testhost**
  - **This would then be sent in the following forms to see what effect they have:**
- **mailmailmailmail from: sender@testhost**
- **mail fromfromfromfrom: sender@testhost**
- **mail from:::: sender@testhost**
- **mail from: sendersendersendersender@testhost**
- **mail from: sender@@@@testhost**
- **mail from: sender@testhosttesthosttesthosttesthost**

Fuzzing Apps

Example: fuzzing a pdf viewer

- **Google for .pdf (about 1 billion results)**
- **Crawl pages to build a corpus**
- **Use fuzzing tool (or script to)**
  1. **Grab a file**
  2. **Mutate that file**
  3. **Feed it to the program**
  4. **Record if it crashed (and input that crashed it)**

Fuzzer Classes

- **Black-box fuzzing**
  - **Treating the system as a blackbox during fuzzing.**
  - **not knowing details of the implementation**
- **Grey-box fuzzing**
  - **Instrumentation to SuT allowed.**
- **White-box fuzzing**
  - **Design fuzzing based on internals of the SuT**

Fuzzing: Black-Box

- original idea of Professor Miller 1988
- Pros: Easy to configure
- Cons: May not search efficiently
  - May re-run the same path over again (low coverage)
  - May be very hard to generate inputs for certain paths (checksums, hashes, restrictive conditions)

```
function( char *name, char *passwd, char *buf )
{
      if ( authenticate_user( name, passwd )) {
            if ( check_format( buf )) {
                  update( buf ); // crash here
            }
      }
}
```

  - May cause the program to terminate for logical reasons

Example: fail format checks and stop

Fuzzing: Black-Box

Mutation-Based Fuzzing, more in Chapter 5

- **User supplies a well-formed input**
- **Fuzzing: Generate random changes to that input**
- **No assumptions about input**
- **Only assumes that variants of well-formed input may be problematic**
- **Tools:** Taof, GPF, ProxyFuzz, FileFuzz, Filep, etc.

Example: zzuf - http://sam.zoy.org/zzuf/

```
zzuf -s 0:1000000 -c -C 0 -q -T 3 objdump -x win9x.exe
```

- **Fuzzes the program objdump using the sample input win9x.exe**
- **Try 1M seed values (-s) from command line (-c) and keep running if crashed (-C 0) with timeout (-T 3)**

Fuzzing: Black-Box

Mutation-Based Fuzzing

- **Pros:**
  - **Easy to setup, and not dependent on program details**
    - **But may be strongly biased by the initial input**
- **Cons:**
  - **May re-run the same path over again (same test)**
  - **May be very hard to generate inputs for certain paths (checksums, hashes, restrictive conditions)**

Fuzzing: Black-Box

Generation-Based Fuzzing

- **Generational fuzzer generates inputs "from scratch" rather than using an initial input and mutating**
- **However, require the user to specify a format or protocol spec to start**
- **Equivalently, write a generator for generating well-formatted input**

Examples include: SPIKE, Peach Fuzz

- **However format-aware fuzzing is cumbersome, because you'll need a fuzzer specification for every input format you are fuzzing**

Fuzzing: Black-Box

Generation-Based Fuzzing

- **Can be more accurate, but at a cost**
- **Pros: More complete search**
  - **Values more specific to the program operation**
  - **Can account for dependencies between inputs**
- **Cons: More work**
  - **Get the specification**
  - **Write the generator – ad hoc**
  - **Need to do for each program**

Fuzzing: Grey-Box

- Finds flaws, but still does not understand the program
- Pros: Much better than black box testing
  - Essentially no configuration
  - Lots of crashes have been identified
- Cons: Still a bit of a stab in the dark
  - May not be able to execute some paths
  - Searches for inputs independently from the program
  - Need to improve the effectiveness further

Fuzzing: Grey-Box

Coverage-Based Fuzzing

- **Rather than treating the program as a black box, instrument the program to track coverage**
  - **E.g., the edges covered**
- **Maintain a pool of high-quality tests**
- **Start with some initial ones specified by users**
- **Mutate tests in the pool to generate new tests**
- **Run new tests**
- **If a new test leads to new coverage (e.g., edges),**
  - **save the new test to the pool;**
  - **otherwise, discard the new test**

Fuzzing: Grey-Box

Coverage-Based Fuzzing

- **Special type of mutation-based fuzzing**
- **Run mutated inputs on instrumented program and measure code coverage**
  - **Examples: AFL, libfuzzer**
- **Search for mutants that result in coverage increase**
- **Often use genetic algorithms, i.e., try random mutations on test corpus and only add mutants to the corpus if coverage increases**

Fuzzing: Grey-Box/Coverage-Based

Coverage-Based Fuzzing, example tool

- **American Fuzzy Lop (AFL)**
- **"State of the practice" at this time**

American Fuzzy Lop Rabbit ?

Fuzzing: Grey-Box/Coverage-Based

American Fuzzy Lop (AFL), Build

- **Provides compiler wrappers for gcc to instrument target program to track test coverage**
- **Replace the gcc compiler in your build process with afl-gcc**

For example, in the Makefile

- **CC=path-to/afl-gcc**
- **Then build your target program with afl-gcc**
- **Generate a binary instrumented for AFL fuzzing**

Fuzzing: Grey-Box/Coverage-Based

American Fuzzy Lop (AFL), Setting up

- **Compiling through AFL**
  - **Basically, replace gcc by afl-gcc**
  - **path-to-afl/afl-gcc test.c -o test**
- **Fuzzing through AFL**
  - **path-to-afl/afl-fuzz -i testcase -o output ./test @@**
  - **Assuming test cases are under testcase, the output goes to the output dir**
  - **@@ tells AFL to take the file names under testcase and feed it to test**

Fuzzing: Grey-Box/Coverage-Based

American Fuzzy Lop (AFL), Setting up the environment

- **After you install AFL but before you can use it effectively, you must set the following environment variables**

E.g., On CentOS

```
export AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1
export AFL_SKIP_CPUFREQ=1
```

- **The former speeds up response from crashes**
- **The latter suppresses AFL complaint about missing some short-lived processes**

Fuzzing: Grey-Box/Coverage-Based

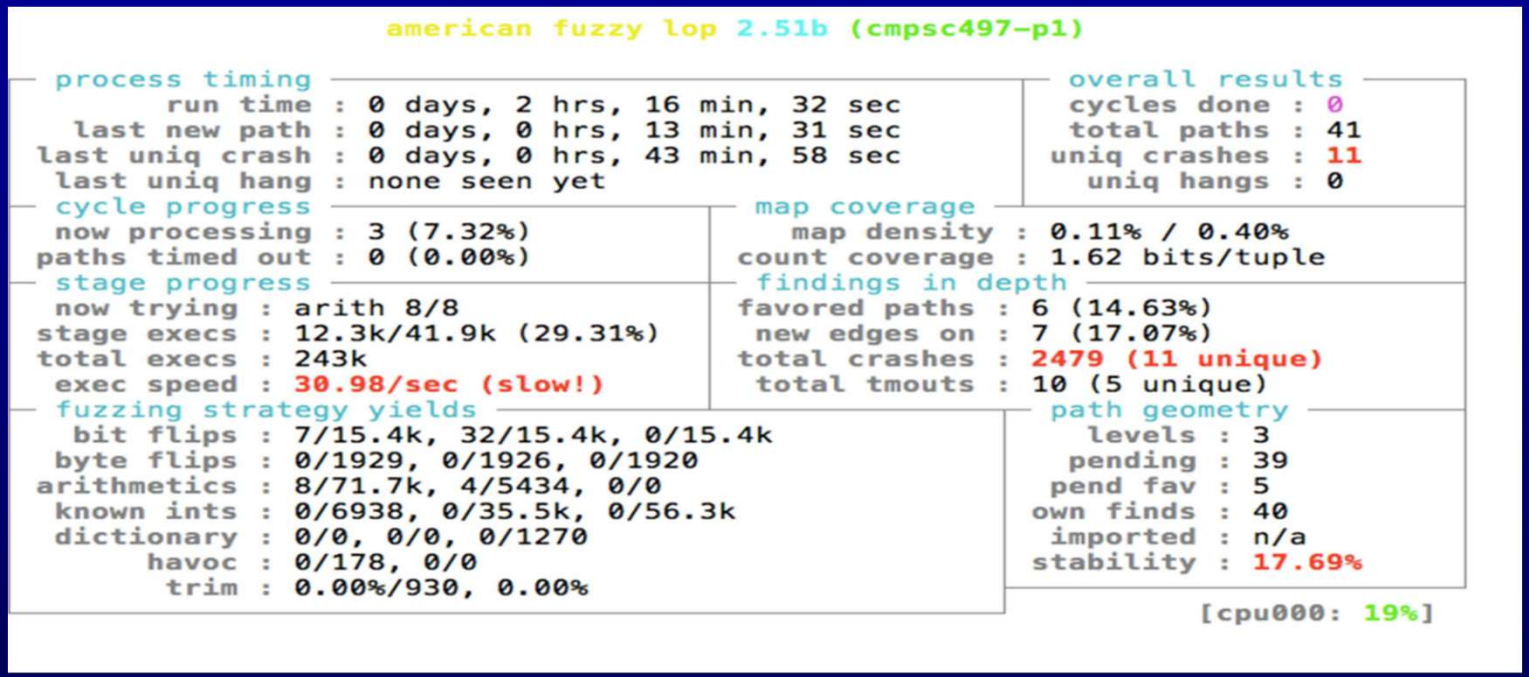
AFL, configuration

- **Instrument the binary at compile-time**
- **Regular mode: instrument assembly**
- **Recent addition: LLVM compiler instrumentation mode**
- **Provide 64K counters representing all edges in the app**
- **Hashtable keeps track of # of execution of edges**
  - **8 bits per edge (# of executions: 1, 2, 3, 4-7, 8-15, 16-31, 32-127, 128+)**
  - **Imprecise (edges may collide) but very efficient**
- **AFL-fuzz is the driver process, the target app runs as separate process(es)**

Fuzzing: Grey-Box/Coverage-Based

AFL Display

- Tracks the execution of the fuzzer



- Key information are
  - "total paths" – number of different execution paths tried
  - "unique crashes" – number of unique crash locations

Fuzzing: Grey-Box/Coverage-Based

AFL Output

- **Shows the results of the fuzzer**
  - **E.g., provides inputs that will cause the crash**
- **File "fuzzer_stats" provides summary of stats – UI**
- **File "plot_data" shows the progress of fuzzer**
- **Directory "queue" shows inputs that led to paths**
- **Directory "crashes" contains input that caused crash**
- **Directory "hangs" contains input that caused hang**

AFL Crashes

- **May be caused by failed assertions – as they abort**
- **Had several assertions caught as crashes, but format violated my checks**

Fuzzing: Grey-Box/Coverage-Based

AFL Operation

- **How does AFL work?**

  http://lcamtuf.coredump.cx/afl/technical_details.txt

- **Mutation strategies**
  - **Highly deterministic at first – bit flips, add/sub integer values, and choose interesting integer values**
  - **Then, non-deterministic choices – insertions, deletions, and combinations of test cases**

Fuzzing: Grey-Box/Coverage-Based

- Finds flaws, but still does not understand the program
- Pros: Much better than black box testing
  - Essentially no configuration
  - Lots of crashes have been identified
- Cons: Still a bit of a stab in the dark
  - May not be able to execute some paths
  - Searches for inputs independently from the program
- Need to improve the effectiveness further

Fuzzing: Grey-Box

Dataflow-guided fuzzing

- **Intercept the data flow, analyze the inputs of comparisons**
- **Incurs extra overhead**
- **Modify the test inputs, observe the effect on comparisons**
- **Prototype implementations in libFuzzer and go-fuzz**

Fuzzing: White-Box

- Combines test generation with fuzzing
  - Test generation based on static analysis and/or symbolic execution
  - Rather than generating new inputs and hoping that they enable a new path to be executed, compute inputs that will execute a desired path
    - And use them as fuzzing inputs
- Goal:
  - Given a sequential program with a set of input parameters, generate a set of inputs that maximizes code coverage

Fuzzing Tools

Input Generation

- **Existing generational fuzzers for common protocols (ftp, http, SNMP, etc.)**
  - **Mu-4000, Codenomicon, PROTOS, FTPFuzz**
- **Fuzzing Frameworks: You provide a spec, they provide a fuzz set**
  - **SPIKE, Peach, Sulley**
- **Dumb Fuzzing automated: you provide the files or packet traces, they provide the fuzz sets**
  - **Filep, Taof, GPF, ProxyFuzz, PeachShark**
- **Many special purpose fuzzers already exist as well**
  - **ActiveX (AxMan), regular expressions, etc.**

Fuzzing Tools

Input Inject

- **Simplest**
  - **Run program on fuzzed file**
  - **Replay fuzzed packet trace**
  - **ZZUF, Taof, GPF, ProxyFuzz, FileFuzz, Filep, etc**
- **Modify existing program/client**
  - **Invoke fuzzer at appropriate point**
- **Use fuzzing framework**
  - **e.g. Peach automates generating COM interface fuzzers**

Fuzzing Tools

Test Oracle – failure detection

- **See if program crashed**
  - **Type of crash can tell a lot (SEGV vs. assert fail)**
- **Run program under dynamic memory error detector (valgrind/purify/AddressSanitizer)**
  - **Catch more bugs, but more expensive per run.**
- **See if program locks up**
- **Roll your own checker e.g. valgrind skins**

Fuzzing Tools

Workflow Automation

- **Sulley, Peach, Mu-4000 all provide tools to aid setup, running, recording, etc.**
- **Virtual machines can help create reproducable workload**
- **Some assembly still required**